

Output Preserving Acceleration of the Bad Patch Finder in the Scrollreading Pipeline

Giovanni Pellerano

Milano, Italy

giovanni.pellerano@evilaliv3.org

Abstract—The bad patch finder of Will Stevens’ scrollreading chain discards pieces of surface that cannot sit together, and on three tangled growths of PHerc. 1447 it ran for hours and was abandoned. We made two changes inside its one function. The first stops the enumeration from extending a chain that is already dead, and the second replaces a map rebuilt on every pass with a vector. On a quiet machine the *c* stage of the tangle seed40 goes from 7,094.8 s to 35.5, about two hundred times faster. On seed38 it goes from 7,930.6 s to 61.2, a factor that a disturbance of the untouched run can only have inflated. On a ribbon, where the enumeration was never the cost, the factor is only 1.05. Every file the five delivery stages write is byte identical between the arms on fifteen named growth trees of two scrolls, and the stage’s whole standard output is identical on fourteen of them. A third change, a guard, turns an abort into a result on a tree whose alignment file names missing patches. The changes make a search over many seeds affordable but move no delivered square. The main limit is that identity is not correctness.

The organisers of the Vesuvius Challenge ask how to make the reading of a scroll work “automatically, reliably, and at scale for every scroll” [1]. Will Stevens’ program has a step that finds the pieces of surface that cannot sit together and throws them away. On three tangled growths of PHerc. 1447 that step ran for hours and was abandoned before a single sheet came out. Measured against his program as published, on two scrolls, PHerc. 1447 and PHerc. 0139, our changes make it finish quickly and leave every file it writes identical. Fig. 1 shows where those abandoned tangles lay: in the air around the scroll, while on the seeds that grew on the papyrus the step was never the slow one. A step that finishes on every tree is what lets the chain run over a whole scroll with nobody watching it, and whoever reads a scroll needs whole sheets before there is anything to read.

Our contribution is two changes to that step, S1 and S2, with the evidence that they change no byte the chain writes. We add a guard, S3, that lets the step finish on a tree where it aborted. The program, its stages and the growth that feeds them are Stevens’ [2]. The arm we compare against already carries an earlier performance series of this project, which section 4 remeasures. The assembly of this chain on PHerc. 0826 is measured in a separate article sent with this one [3].

What this work is about, in plain words

The scrolls from Herculaneum were burnt by a volcano. They cannot be unrolled by hand, so they are scanned, and software has to find the sheets of papyrus inside the scan. Each sheet is then flattened so the writing on it can be read.

This work is about one program in that chain, written by Will Stevens [2]. The program first grows many small pieces of surface, called patches. A later step looks for patches that

cannot be joined to their neighbours consistently, and throws those away.

On three of the growths that step did not finish, and our runner stopped it at its time limit. Two changes inside it make the same step finish in about a minute. Nothing it produces changes: every file the chain writes afterwards is identical, byte for byte, to the file it wrote before.

A third change repairs a crash. On one growth the program stopped with an error because its own bookkeeping named pieces that were not on disk. It now refuses those pieces and says how many it refused, instead of stopping.

A reader can check all of this. Every number in this work points to the file it was read from, through a marker and the appendix, and every one of those files is in this folder. The claim that nothing changed is a comparison of every file the chain writes, not of the finished pictures alone. Where a measurement was taken on a busy machine, the text says so.

I. The chain and its stages, on two scrolls

This section introduces the chain, the three changes and the two scrolls, and shows that once the changes are in, the *c* stage is no longer the whole cost of a run.

The object is the patch based surface tracing chain of Will Stevens [2], `pipeline9/simpaper10` of the scrollreading repository at commit 62cbc21, which is `refs/heads/main` of the clone on this disk [1]. The copy of `simpaper10.cpp` the builds here were laid from hashes to the same blob as 62cbc21: `pipeline9/simpaper10.cpp`. A run is a growth, `g 36000`, followed by five delivery stages: `c`, `l`, `vm 10`, `hm 10`, `fm 30 10`. The growth writes patches and an alignment file, `rel.csv`, one line per alignment between two patch numbers. The *c* stage, the bad patch finder, reads them both, enumerates chains of patches of length 2 to 5, scores them, and condemns the patches that cannot be placed consistently.

A. The three changes this work proposes, and how they are named here

The series these patches belong to numbers them on disk, and those numbers mean nothing to a reader. This work calls them S1, S2 and S3, in the order the text meets them, and the serial appears only in the last column.

The prose below says S1, S2 and S3. The earlier performance patch is in the table because this work remeasures both of the things it claims, its factor and its identity. Neither could be read back from anything on this disk when the work was begun. It is not one of the three, and the prose never counts

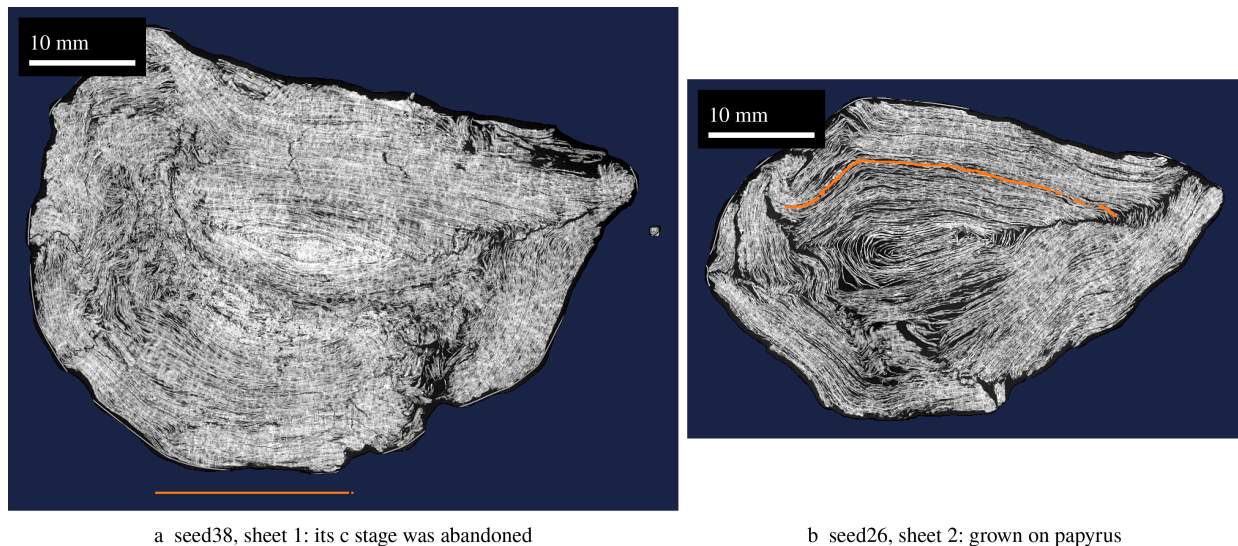


Fig. 1. **A sheet the old stage abandoned, on the scroll it was grown in.** Each panel is one horizontal slice of the scan of PHerc. 1447, seen from above. Papyrus is grey, and dark blue is outside the scan's mask, where there is no scroll. Orange is where a delivered sheet crosses the slice. Panel a is the largest sheet of the trees whose untouched bad patch finder the seed runner stopped, seed38's sheet 1: it is a straight wall in the air beside the scroll. Panel b, for comparison, is the largest sheet of the seeds that grew on papyrus, seed26's sheet 2, which follows a turn of the papyrus. The white bar is 10 mm.

it among them. Its identity was measured on 2026-09-22, file by file and on the printed output ^[2].

All three changes are now on the author's repository, where they were opened on 22 September 2026. S1, S2 and S3 are one pull request, WillStevens/scrollreading #4, which is stacked on #3: that one carries the earlier performance patch alone, because the three do not apply to main without it. The defect behind the crash of section 5, a growth started into a folder that already holds one, is issue #2 on the same repository.

Two scrolls are used, and they are not interchangeable. The voxel of PHerc. 1447 is 8.640 um, the scroll's own value from the dataset manifest ^[68]. PHerc. 0139, whose voxel is 9.362 um, is used here only as a second body of growth trees on which byte equality and stage times can be checked.

The next table gives what the five stages cost on the arm carrying the two changes of section 4, at four OpenMP threads ^[69]. Only the c row was timed on a quiet machine. The other four were timed on a machine shared with other studies, with 7.2 to 8.4 of its 24 cores busy.

The c row is the median of three runs on a machine doing nothing else ^[3]. On the ribbon the three runs took 47.4, 48.3 and 48.6 s. On the tangle they took 34.5, 35.5 and 35.6 s. Their spreads are 2.53 and 3.19 per cent of the smallest run ^[21]. The four rows below the c row are single timings from the earlier harness on a shared machine and have not been repeated quietly. The column therefore mixes two kinds of measurement, and the table says which is which rather than presenting five numbers of equal weight.

What matters in this table is the proportion. The four stages after c cost 17.0 seconds on the tangle and 18.9 on the ribbon, summed from the same file. Before the changes, the c stage of that same tangle took 7,094.8 s on the quiet bench ^[21], so it was not one stage among five: it was the run. The growth that precedes them takes 2,400 to 10,075 s over the nine trees of section 2 ^[5]. Those growths ran two or three at a time beside

each other, which makes their wall clocks a cost record and not a result.

II. Ten drawn seeds on PHerc. 1447

This section shows that the growth trees of PHerc. 1447 fall into two kinds, ribbons and tangles. The kind predicts the cost of the c stage but not the size of the sheet the chain delivers.

Ten seed points were drawn on PHerc. 1447 and grown ^[6]. Nine of the ten finished their growth by themselves and were delivered and measured. seed11's growth returned 3, so it is counted apart in every column and never folded into a median with the others.

The fan out of a tree is the average number of neighbours a patch has in its alignment map. It splits the ten trees into two groups with nothing in between. Ribbons grew as one long strip and are cheap for the c stage. Tangles grew folded on themselves and are expensive. AugmentAlignmentMap inserts the reverse of every alignment, so the map the chain walk sees is symmetric, and its mean fan out is twice the edge count over the number of keys ^[7].

One prediction was written before its answer existed, and it was right. The kind of a tree can be read from the growth's own log long before the growth ends, and seed44 is the one seed where that rule was applied in advance rather than fitted. The reading was taken 532 seconds into a growth that then ran 9,097 s. By then the share of the first 100,000 log lines stood at 0.11963, against a ribbon band that topped out at 0.04275. The prediction recorded was tangle. Its note said that the share sat in the gap between the two bands, 0.00039 below the nearest tangle rather than inside the tangle band ^[8]. The answer row, appended when the growth ended, reads tangle at fan out 32.97. The rule has since gained a repeat control it did not have when it was written. The same seed was grown twice, under different machine states, to logs of 4,010,300 and 3,348,977 lines. The two shares differ by 0.00005, against a gap of about 0.07 between the kinds ^[9].

	what it is	what it does	measured effect	identity	on disk
S1	a dead prefix is not extended	when a chain dies the odometer advances at the position it died, instead of enumerating every suffix of a prefix already rejected	with S2, the c stage of the tangle seed40 goes from 7,094.8 s to 35.5, 199.9 times, and of seed38 from 7,930.6 to 61.2	every file the five stages write, byte for byte, on fifteen named trees	corrections/ 00008
S2	the cover's frequency table is a vector	the greedy cover rebuilt a <code>std::map</code> of counts on every pass; patch numbers are small integers, so one cleared vector holds the same counts	measured with S1 above; on a ribbon the pair is 1.05 times, which the work states rather than averages away	the same	corrections/ 00009
S3	a chain through a patch with no geometry is refused	the alignment map has keys the patches map does not; the stage aborted on the first such chain, and the guard counts and refuses it at the point the chain is assembled	turns an abort into a run on the one tree where it happens, and costs time on three of the seven ribbons of the reference scroll	byte identical on the three trees with no such patch	corrections/ 00007
<i>the earlier performance patch</i>	the selection render state is cleared per touched cell and the normals are pre-computed	it is not one of the three and is listed here because this work remeasures it: its own study no longer exists	31.11 times on the ribbon of 9,512 patches, median against median	the whole downstream on fresh copies of two growth trees, one of each scroll: 94,962 files compared, none differing, and the only difference in anything either arm prints is one line, points skipped as off-grid: 0, which only the arm with the patch writes	performance/ 00002

stage	seed26, a ribbon, 9,512 patches	seed40, a tangle, 14,959 patches	machine
c	48.3	35.5	quiet, 0.05 to 0.12 cores busy of 24
l	2.5	5.3	shared, 7.2 to 8.4
vm 10	2.5	5.4	shared, 7.2 to 8.4
hm 10	0.1	0.1	shared, 7.2 to 8.4
fm 30 10	13.8	6.2	shared, 7.2 to 8.4

The measure of a delivered sheet used throughout is its square: the side of the exact largest axis aligned, fully covered square in the sheet, at that scroll's own voxel. Over the three seeds on papyrus, the largest square is 12.7244 mm, on seed26, and the smallest is 10.3909 mm, on seed01. The median is 11.3635 mm, on seed15^[71]. Those three seeds hand back thirty delivered sheets, ten per seed, and none of them reaches 20 mm^[72]. The other seven seeds, seed11, seed34, seed35, seed38, seed40, seed44 and seed48, are the seven tangles above, and they grew in air: the raw masked scan is absent at each of their seed points^[73].

The fan out does not bound the square, and seed44 is the case that shows it. Nine seeds have both a fan out and a square. The three ribbons hold three of the four largest squares,

12.7244, 11.3635 and 10.3909 mm. Five of the six tangles are at or below 9.2948^[10]. The sixth tangle is the counterexample, and it is the very seed the rule was applied to in advance. seed44, a tangle at fan out 32.97, delivers 11.1968 mm, the third largest square of the nine. That is above the ribbon of seed01, at 10.3909, and below the other two ribbons.

A bound had been registered before seed44's downstream ran: a seed at or above fan out 27.40 gives a square at or below 9.2948 mm. The file called it "likely, and it is the sixth test of a bound that is five for five"^[11]. The delivered square is above it. The study recorded its own refutation in that file on 2026-09-22T02:23:46Z, as a second row appended beside the prediction rather than as an edit of it, because a preregistration is not rewritten. The prediction row's answer column still reads

kind	seeds	fan out
ribbon	seed26, seed01, seed15	4.51, 4.61, 4.64
tangle	seed38, seed40, seed48, seed44, seed11, seed34, seed35	27.40, 30.82, 30.86, 32.97, 33.97, 35.88, 37.67

“not yet known, the downstream has not been run”, which is what it said when it was written. The appended row’s `answer` reads “WRONG on both”. The square came out at 11.1968 mm against a bound of 9.2948. The traced area came out at 4,292.4 mm² against a bound of 1,898.5. The fan out, in short, orders the cost of the `c` stage sharply, which is the subject of section 3, and it does not bound the square.

III. Where the `c` stage’s time goes

This section shows where the untouched `c` stage spends its time: in the descent of a `std::map`, driven by an enumeration whose size grows with the fourth power of the fan out.

A profile was taken before anything was changed, with `perf record -F 299 -e cycles:u`, over the first 2,676.3 s of the untouched stage on a fresh copy of seed40’s tree. The run was stopped by a signal at that point, and its return code reads -15^[12]. The profile file carries the shares and not the duration. The binary carried `-g` and nothing else: its `.text` section is byte for byte that of the same build without `-g`^[13], so the profile is of the code that ships.

About eighty per cent of the stage, then, is the descent of a `std::map`, and the line of the chain’s own source that drives it is the odometer’s validity walk.

To see why, it helps to count the work instead of timing it. The enumeration is an odometer over `indices[0..length-1]`, a counter that walks every run of patches in turn. The number of index tuples it visits for a chain of length `L` is the number of walks of `L` minus 1 edges in the augmented alignment multigraph. That count can be taken exactly from a tree’s own `rel.csv` and `patches/`, with no run at all^[14]. A counting build of the stage itself, with one increment per visit of the `while` body, agrees with it to the step: on seed26 both read 10,761,063 over the four rounds^[15]. The count was taken this way because the untouched binary, given the same tree at four threads, had not begun the round at length 5 after 3,000 s^[16].

seed40 totals 51,500,264,151 steps, and the round at length 5 is 98.05 per cent of them^[17]. The walk grows with the fourth power of the fan out at the longest length the stage runs. The file of fan outs states that as a cost model of the walk in the source, not as a measured time^[74]. The exact counts above are the measurement, and the first two rows are the ones a reader should compare.

IV. Two changes and their proof

This section describes the two changes, shows that they make the `c` stage finish on the tangles, and shows that every byte the chain writes stays the same.

Both are patch files of this project’s series for that repository, applied after the orphan guard of section 5. Neither changes a decision the stage makes; both change how much work it does to reach the same decision.

The first change, S1, stops the enumeration from extending a prefix that is already dead. When the chain being built meets a repeated patch or a patch already condemned, the build loop breaks, but the odometer still advances its last index. Every suffix of a dead prefix is therefore enumerated one at a time, and rejected again for the reason the prefix was rejected for. The change records the position at which the chain died and advances the odometer there, resetting the indices below it to zero, which is where the carry would have left them. A bad first patch is the same argument at depth zero.

The second change, S2, makes the frequency table of the cover a vector. The greedy cover at the end of the same function condemns one patch per pass, and on every pass it rebuilds a `std::map<int,int>` of counts from nothing. Patch numbers are small non-negative integers, so one vector indexed by patch number, cleared at the top of each pass, holds the same counts with no allocation and no tree descent. The scan order and the comparison are untouched, so the patch chosen on each pass is the same patch.

The build script laid down a working copy from the pinned upstream plus the series with both patches. It is byte for byte the working copy laid with the guard alone and then edited by the two change scripts^[18]. The patch files, and not a hand edited tree, are what was built and measured.

The first change shows in the step counts before it shows in any clock. On seed40 the counting build with S1 takes 946,327 steps, against 51,500,264,151 before^[19]. That is a factor of 54,421 (Fig. 2), and the last row of the table of section 3 gives the four rounds. On seed26 the two variants print the same number of kept chains at every length, which is the change’s correctness read off the instrument itself. On the two shortest lengths those counts are 21,432 and 52,571. On the two longest they are 191,087 and 682,561. Neither number depends on the machine.

With the enumeration pruned, the time moved elsewhere. A diagnostic build timed the three phases of the function with `omp_get_wtime` on seed40’s tree^[20].

After the first change the enumeration this work set out to fix costs 0.46 s, and the greedy cover costs 252.96 s. That is why there is a second change, and why it is the one it is.

The next table gives the `c` stage on six seeds before the changes, with S1, and with both, at four threads for the arms^[75]. Both changed arms also carry the guard S3. Its times were taken on a shared machine, which is the weaker kind of measurement. The quiet bench has since retaken three of these seeds with nothing else running, and wherever a figure of this table is quoted elsewhere in this work, it is the quiet one.

Three of those rows have since been taken again on a machine that was doing nothing else, and the pair that matters most is one of them.

Every run in that table used four threads, one run at a time, with 0.05 to 0.14 cores busy of 24 before each^[21]. On seed40 the quiet pair is 7,094.8 seconds against 35.5, about

share	what	sort
91.50 %	BadPatchFinder::	symbol
50.98 %	FindBadPatchesGeneral	
	stl_tree.h:1437, the right child step	source line
	of a red black descent	
8.14, 7.21, 4.76, 3.95 %	stl_tree.h:2604, 2605, 2603,	source line
	2607, the body of _M_lower_bound	
5.41 %	stl_tree.h:1425, the left child step	source line
3.20 %	badpatchfinder.cpp:426, the	source line
	odometer's rewalk of the chain	

tree	length 2	length 3	length 4	length 5
seed26, fan out 4.51	42,868	243,529	1,443,677	9,030,989
seed40, fan out 30.82	460,985	20,319,828	983,163,150	50,496,320,188
seed40 with S1 of section 4, counted	460,985	73,846	130,903	280,593

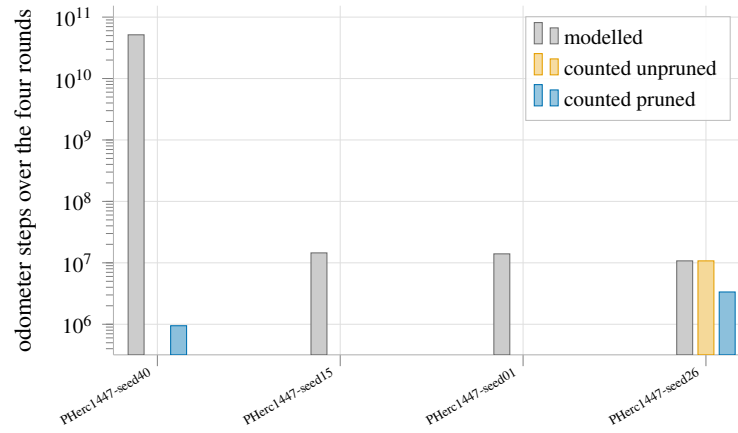


Fig. 2. **The odometer's steps, modelled against counted.** What the change does is not run faster, it is do less. On the tangle seed40 the enumeration as written takes 51,500,264,151 steps and the changed one 946,327, a factor of 54,421, and that is a count of work and not a clock. The three ribbons sit three and a half orders of magnitude below the tangle before anything is changed. That is why the same change is worth about two hundred times on one and a factor of 1.05 on the other.

arm	setup	odometer	sequences	cover	stage
S3 and S1	1.49	0.46	15.41	252.96	277.0 s
S3, S1 and S2	1.69	0.49	15.42	15.58	40.1 s

two hundred times (Fig. 3). The factor is 199.9 taking the median of each arm, which is the shape every other factor in this work uses, and 205.65 taking the smallest of each [22]. On seed38 the quiet pair is 7,930.6 seconds against 61.2, a factor of 129.6, which the next paragraph qualifies. The shared machine's ratio on seed38 is 123.9, and on seed48, which the quiet bench did not retake, it is 152.6 [75].

One caveat belongs here, and the bench's own record of what else ran states it run by run [76]. None of the changed runs shared the machine with anything. Three of the untouched runs did. On seed26 the second untouched run, 49.4 s, had a relaunch of an unrelated mesh tool on one core for 24 s of it, 0.486 of its time, while the first, 51.8 s, had nothing. On seed40 the untouched run had the same kind of burst for 24 s, 0.003 of its time. On seed38 the untouched run overlapped the coordinator's read back sweep for 1,572 s, 0.198 of its time, at up to one core. A disturbance can only slow the run it falls in, so on the untouched arm it can only inflate a factor. On seed40

it cannot move a factor of two hundred. On seed38 the factor of 129.6 is an upper bound, and the shared machine's ratio on that tree, 123.9, is below it. On seed26 the disturbed run is the faster of the two, so the burst did not slow it measurably.

A seventh tree, seed11, is the one where both binaries finished and nothing had to be inferred from a stopped run. It has 26,947 patches and 457,647 relation rows [23]. The untouched stage took 22,699 s on it, 6 hours 18 minutes, and returned zero [24]. The five stages together took 22,775 s. The changed stage took 128.0 s on a copy of the same tree [25]. Both timings were taken on a shared machine.

The reference scroll tells the other half of the story, because every tree grown on it is a ribbon. The same pair of arms was run four times per arm on each of seven distinct growth trees of PHerc. 0139. Their fan out is only 4.7074 to 4.9490 [26], so the enumeration the first change attacks is cheap on all of them, and there is nothing for it to save [27]. The verdict on each tree calls a difference nothing when the two arms'

seed	rel.csv lines	before	S1	S1 and S2	note
seed01	25,505	30 s	57.9	55.1	before taken at 24 threads, not comparable
seed15	26,476	32 s	85.7	78.3	before taken at 24 threads, not comparable
seed26	21,436	22 s	52.5	51.8	before taken at 24 threads, not comparable
seed38	304,256	7,808 s	605.5	63.0	both arms on a shared machine
seed40	230,513	7,251 s	289.1	34.9	both arms on a shared machine
seed48	284,922	8,300 s	472.7	54.4	both arms on a shared machine

tree	arm	runs	seconds	spread
seed26, 9,512 patches	untouched	2	51.8 and 49.4	2.4 s, 4.86 per cent of the smallest
seed26	with both changes	3	47.4, 48.6, 48.3	1.2 s, 2.53 per cent
seed40, 14,959 patches	untouched	1	7,094.8	not measurable, one run
seed40	with both changes	3	35.6, 35.5, 34.5	1.1 s, 3.19 per cent
seed38, 22,205 patches	untouched	1	7,930.6	not measurable, one run
seed38	with both changes	3	62.2, 61.2, 56.6	5.6 s, 9.89 per cent

observed ranges intersect ^[77].

So on a scroll that produces no tangle the change costs nothing and sometimes gains. Four of the seven trees are not separable from the spread and three are faster, and no tree is slower than its own spread allows. Those seven pairs were measured within one run of one harness but on a machine that was not quiet throughout, and they are not in the quiet bench's order.

A third arm shows something the pair above hides, and the work would be dishonest without it. The harness ran a third binary on every one of those trees: the untouched one plus the orphan guard of section 5, and nothing else ^[28]. The guard adds a check the untouched binary does not make, and on this scroll it costs time.

In this table a ratio above one means the second arm of the pair is the faster ^[29]. On three of the seven trees the guard alone is slower than the untouched binary by more than either arm's spread. It costs 1.33 times on the smallest tree. On C and D150 it costs 1.29 and 1.07. Measured against the guard rather than against the untouched binary, the two changes pay that cost back and more on A, C, D150 and P2. The ranges overlap on none of the four. They gain 1.379 on A and 1.351 on C. On D150 and P2 they gain 1.379 and 1.122.

That is the shape a reader should take away for a scroll of ribbons: the guard is a price, the two changes cover it, and the three together are between free and a gain. The price is worth naming because the guard is the piece that turns an abort into a result, and section 5 is where that trade is argued.

The two changes were accepted on one criterion: the output does not move. We call two runs byte identical when they wrote exactly the same bytes in every file. The criterion covers everything the five stages write, not the delivered sheets alone, on a fresh copy of each tree, compared against sha256 taken before this work compiled anything.

The work claims these numbers and no more. The same file on the reference scroll carries a second sum, of the carried input, which adds to 1,473,675 where the files written by the downstream add to 936 ^[78]. That second sum is not identity evidence. It counts the carried growth tree, which is hard linked between the arms and therefore identical by construction. The real check of that tree is the pristine fingerprint of each one, taken before any stage ran ^[30].

Eight trees are named on the reference scroll, but only seven are distinct. Two of the eight, the growth of `growth-repeat` and the growth of `reference-b40`, are the same tree. Their `rel.csv` has the same sha256 and the same 19,123 lines over 7,854 patches, because one study grew it and the other repeated that growth and got it back byte for byte. They are counted twice above because the comparison was run twice, and the honest population is seven distinct growths. This project quoted the larger number once and withdrew it. It is named here so that nobody quotes it again.

A stronger test than the files is the whole standard output of the `c` stage, because it would catch a difference the files hide. Line for line, it is identical to the run that wrote the sheets on all six PHerc. 1447 trees in both arms ^[31], and on all eight named PHerc. 0139 trees, seven distinct, in both pairings ^[32]. On seed15 the output is 2,914,773 lines long. It names every chain kept, in the order it is kept, with its score, and every patch condemned, in the order it is condemned. The only lines removed before the comparison are the two per chain length that the guard of section 5 prints and an unguarded binary cannot, and each row counts them.

The arm called untouched here is not stock upstream, and one number this project quoted about it was wrong. That arm carries this project's earlier performance series, including the per thread render state, the parallel loops over pairs and over chains, the precomputed normals and the sparse clear. A

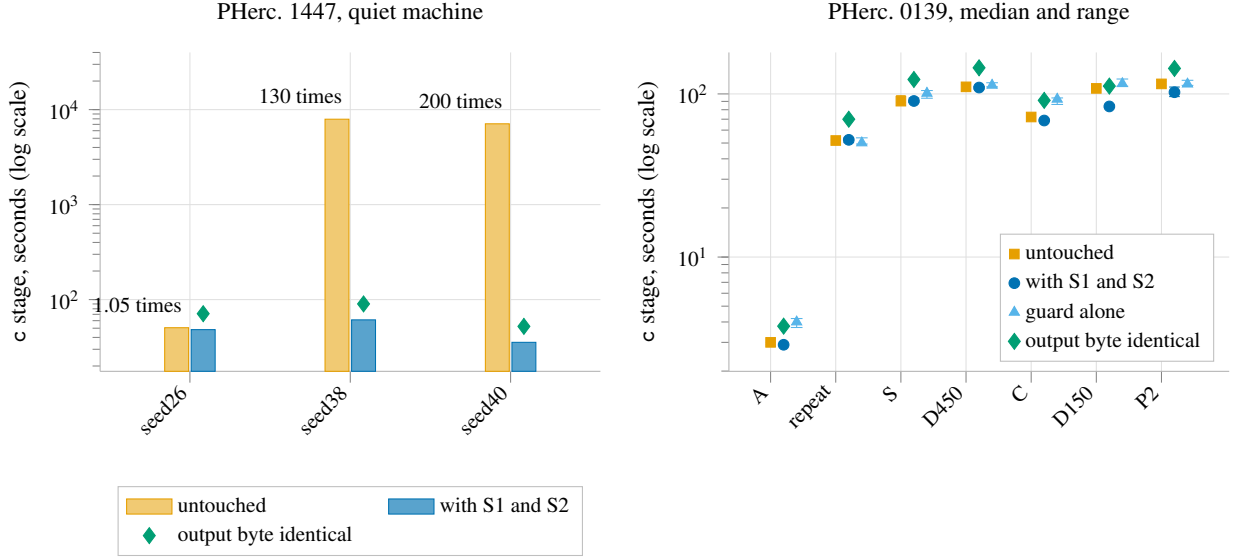


Fig. 3. **The c stage, untouched against changed, on the quiet bench.** On PHerc. 1447 the change is the difference between a stage that finishes and one that is abandoned. The untouched binary takes 7,094.8 s on one tangle and 7,930.6 on the other. The changed one takes 35.5 and 61.2, about two hundred times on seed40 and about a hundred and thirty on seed38. On seed38 the untouched run shared the machine for a fifth of its time, so that factor is an upper bound. On the ribbon, where the enumeration was never the cost, the same pair is 1.05 and the panel shows it at that size rather than hiding it in an average. The three seeds with no quiet pair are left off this panel. Their shared machine before was taken at twenty four threads and their after at four, so the two are not a pair at all. The right panel is the reference scroll, with the median and the observed range of each arm, the guard alone included. A green diamond over a seed or a tree marks output that the evidence shows byte identical between the untouched arm and the arm with both changes, which also carries the guard S3. That covers every file compared and, on PHerc. 1447, the whole standard output of the stage as well.

tree	untouched, median	with both changes, median	per cent	verdict
A, 392 patches	3.0	2.9	-1.7	not separable from the spread
repeat, 7,854	51.8	52.3	+1.2	not separable from the spread
S, 14,583	90.3	90.5	+0.3	not separable from the spread
D450, 15,117	110.5	109.4	-1.0	not separable from the spread
C, 15,231	72.2	68.7	-4.8	c2 is faster
D150, 15,183	108.1	83.8	-22.4	c2 is faster
P2, 15,338	115.2	102.4	-11.1	c2 is faster

prepared text claimed a factor of seventeen for that group, on the strength of a study that no longer exists. The group was remeasured on a quiet machine on the night of 21 September, on seed26, by building an arm with that patch removed [33]. Both ranges are given beside the medians, because with two and three runs a median is a weak statistic.

Two files of that study carry this factor, and they are not the same statistic, so both are named wherever either is used [79]. One compares a median against a median and gives 31.11, the value quoted above. The other compares the smallest run of each arm and gives 31.34 on the same rows. Its terms are 1,548.2 over 49.4. The two differ by less than one per cent, and neither corrects the other: a median and a minimum are different questions about the same five runs.

Every run was four threads on a fresh copy of the tree, one at a time, with the machine between 0.05 and 0.12 cores busy of 24 before each. This is one tree of 9,512 patches, a ribbon, and the file says so in its own header. The seventeen is withdrawn: it cannot be read back from anything on this disk.

On the ribbon, one earlier measurement did not reproduce. A first measurement on a loaded machine showed the changed binary 18 per cent slower on one ribbon, and the quiet remeasurement does not reproduce it. Both sets are in the next table, with the cores busy when each was taken.

On the quiet machine the changed binary is faster on that tree by a factor of 1.05. What moved most between the two sets is the untouched arm, from 39.0 to 39.8 s in the afternoon to 49.4 and 51.8 s at night. The changed runs moved far less, from a median of 46.6 to 48.3 s. Section 7 says what can and cannot be concluded from that.

V. The crash and the guard

This section shows why the untouched stage aborts on one tree, and how the third change, S3, which we call the guard, turns that abort into a result.

On the tree of seed34 the untouched stage aborts. It returns code 134 after 39 s at four threads, with `terminate` called after throwing an instance of `'std:`

tree	untouched over the guard alone	ranges overlap	the guard over both changes	overlap
A, 392 patches repeat, 7,854	0.750	no	1.379	no
S, 14,583	1.034	yes	0.958	yes
D450, 15,117	0.895	yes	1.115	yes
C, 15,231	0.978	yes	1.033	yes
D150, 15,183	0.778	no	1.351	no
P2, 15,338	0.935	no	1.379	no
	1.003	yes	1.122	no

body of evidence	comparisons	differing	file
six growth trees of PHerc. 1447, both arms, everything written	585	0	c-stage-cost/evidence/identity.csv, column byte_identical
a seventh tree of PHerc. 1447, seed11	40	0	c-stage-cost/evidence/identity-seed11-untouched.csv
eight named growth trees of PHerc. 0139, seven distinct, three arm pairings, files the downstream wrote	936	0	c-stage-on-0139/evidence/identity-by-tree.csv, columns written_by_downstream and written_differing

ratio	over	median against median	factor	the two ranges
the earlier performance patch	the series without it over the series with it	1,574.0 over 50.6	31.11	1,548.2 to 1,675.7 over 3 runs, 49.4 to 51.8 over 2
the two changes of section 4	the series with the patch over that plus both changes	50.6 over 48.3	1.05	49.4 to 51.8 over 2, 47.4 to 48.6 over 3
all six changes together	without the patch over with everything	1,574.0 over 48.3	32.59	as above

when	machine, cores busy of 24	untouched	with both changes	file
afternoon of 21 September	6.1 to 8.3	39.0, 39.3, 39.5, 39.8	45.9, 46.6, 47.2	c-stage-cost/evidence/small-tree-cost.csv
night of 21 September, quiet	0.05 to 0.12	51.8, 49.4	47.4, 48.6, 48.3	quiet-bench/evidence/c-stage-runs.csv

:out_of_range', what(): map::at^[35]. The throw is a lookup of the patches map with a key that map does not hold, at line 436 of badpatchfinder.cpp^{[36][37]}.

An instrumented copy of the stage, built to report and never to run the chain, finds 202 failing chains, all at length 2^[37]. Every one of them has the same shape: the alignment map has the key, the patches map does not, the failing element is the second of the chain, and the first element is in both maps.

The missing key is a patch number that `rel.csv` names as the target of an alignment and that has no `patch_<n>.bin` on disk. The loader keys the patches map by the files on disk, and it keys the alignment map by `rel.csv`. `AugmentAlignmentMap` then makes every id ever named as a target a key of the alignment map, with no check that the target was loaded. On seed34's tree, `rel.csv` names 104 such ids^[38]. The tree has 28,054 patch files and 505,213 relation rows. Exactly 73 of those ids can be the second element of a kept chain of length 2, because the enumeration keeps a chain only when its first patch is lower than its last and is itself loaded. The same columns read 0 and 0 on the

three trees whose stage finishes.

The guard was checked against a prediction fixed before it existed. The change sits inside `FindBadPatchesGeneral`. A chain that would have been kept and carries an id with no geometry is counted instead of being pushed, and every other chain is treated exactly as before. The stage prints, once per chain length, the count of chains refused, the count of distinct ids and the ids themselves.

The 73 was fixed from `rel.csv` and the patches/ listing before the guard existed, and the 202 from the instrumented diagnostic build of the same study, also before it existed. What the guard then reported on the same tree^[39]:

The two sets of ids at length 2 are equal, with no id on either side alone^[40]. The stage that used to abort instead ran the length 2 round, its vertex cover, its bad patch selection and two further rounds. It was stopped by its own 14,400 s cap inside the round at length 5, not by memory^[41]. That the work does not fit in four hours on that tree is the cost of section 3 and not of this change.

Elsewhere the guard never fires, and it does not change

chain length	chains refused	distinct ids with no geometry
2	202	73, and the prediction column reads yes
3	178	58
4	83	32

what is written. On the three trees with no orphan it reports `chains=0 patches=0` at each of the four lengths, twelve lines in all [42]. On each of those three, every delivered sheet and every file the downstream writes is byte identical between the guarded arm, the untouched arm and the seed runner's own run [43]. That is 60 sheet comparisons and 234 file comparisons in all.

The guard does cost time. Section 4 gives the three trees of the reference scroll on which it is slower than the untouched binary by more than either arm's spread. The reason is that it adds a `count()` to the hot loop, and on a scroll where the enumeration is cheap that loop is most of the stage. The two changes of section 4 more than cover it, and the trade this section argues is a crash turned into a result, not a free check.

The orphan ids come from a growth started into a folder that still held the points of a stopped run, which is a defect of the growth and not of this stage [80]. Of the 413 ids the growth log says cannot be that run's, 327 were created again by the second run and point at geometry that is not the one the alignment was computed against [49]. They crash nothing, and this work counts them without measuring their effect (Fig. 4).

VI. What this does not change

This section states what the acceleration leaves exactly as it was: every property of the delivered surface, and with it the distance to a 20 mm square.

Nothing in this work moves a square, and the reason is section 4. Every output is byte for byte what it was, so every property of the delivered surface is what it was, and every defect the stage had, it still has. What changed is how long the chain takes to produce the same answer, and on one tree, whether it produces one at all.

The distance to a 20 mm square is unchanged, and it is the honest frame for the whole work. The largest square this chain delivers on PHerc. 1447 is 12.7244 mm, drawn on the material in Fig. 5 [54]. It is taken over the nine seeds whose growth finished: the three on papyrus of section 2 and the six tangles that grew in air. The best published segment of that same scroll reaches 13.5262 mm (Fig. 6) [55]. The chain is 0.8018 mm below it. That gap is inside this project's own repeat spread of 2.6213 mm [83], so it decides nothing either way.

The organisers' reference area is a contiguous 400 mm², which is a 20 mm square, and the 20 mm side is the size the squares here are read against. None of the ninety sheets these nine seeds deliver reaches 20 mm [56]. Nor does any of the twenty one published segments of PHerc. 1447 and PHerc. 0800 measured here [57].

So the acceleration buys search, not surface. It makes it cheap to ask the question on many seeds and it makes one class of tree deliverable at all. It does not bring the answer nearer to 20 mm by a millimetre, and nothing in this work should be read as saying it does.

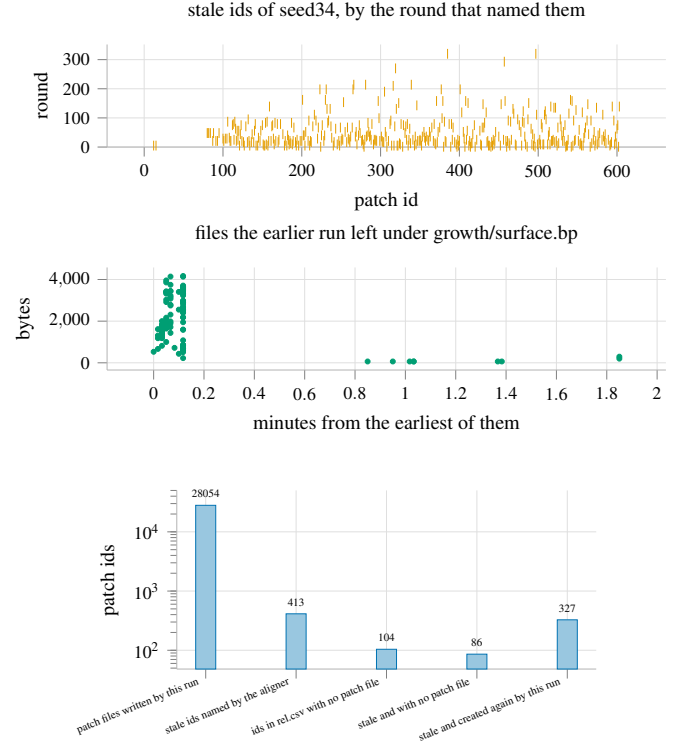


Fig. 4. **Two runs mixed in one folder.** A growth restarted into a folder that already held points does not start again, it adds. The run wrote 28,054 patch files. The aligner's own log names 413 ids that cannot be this run's. 104 ids appear in `rel.csv` with no patch file at all, and 86 of them are among those stale ids. The number that matters is the last: 327 of the stale ids were created again by the second run, so most of the stale ids are invisible in the file listing and show only in the log. The crash of section 5 comes from the ones that are visible.

The author's remedy for patches that cannot sit together is also unchanged by this work, and it is only half on in the chain as we automated it. Of its three methods [4], the bad patch finder runs, while the bridges and the annealing are manual steps that our runner never took [63]. Put back together, the bridges and the annealing change the delivered sheets on 10 of 10 seeds [64]. The annealing draws from the hardware random device, so two runs on one tree are two draws and not a repeat [65], and a seed for it is offered as a correction.

VII. Limits

This section sets out what the evidence does not establish. One caveat bears on the factor on the ribbon. On seed26 the untouched binary read 39.0 to 39.8 s on a loaded afternoon and 51.8 s on a quiet night. The changed binary's median moved far less, from 46.6 to 48.3 s, and the cause is not known.

The mitigation is the one the bench has. The two arms of a ratio are measured back to back, on the same machine in the same state, so a common cause cancels. What does not cancel is comparing a number taken on one day with a number taken

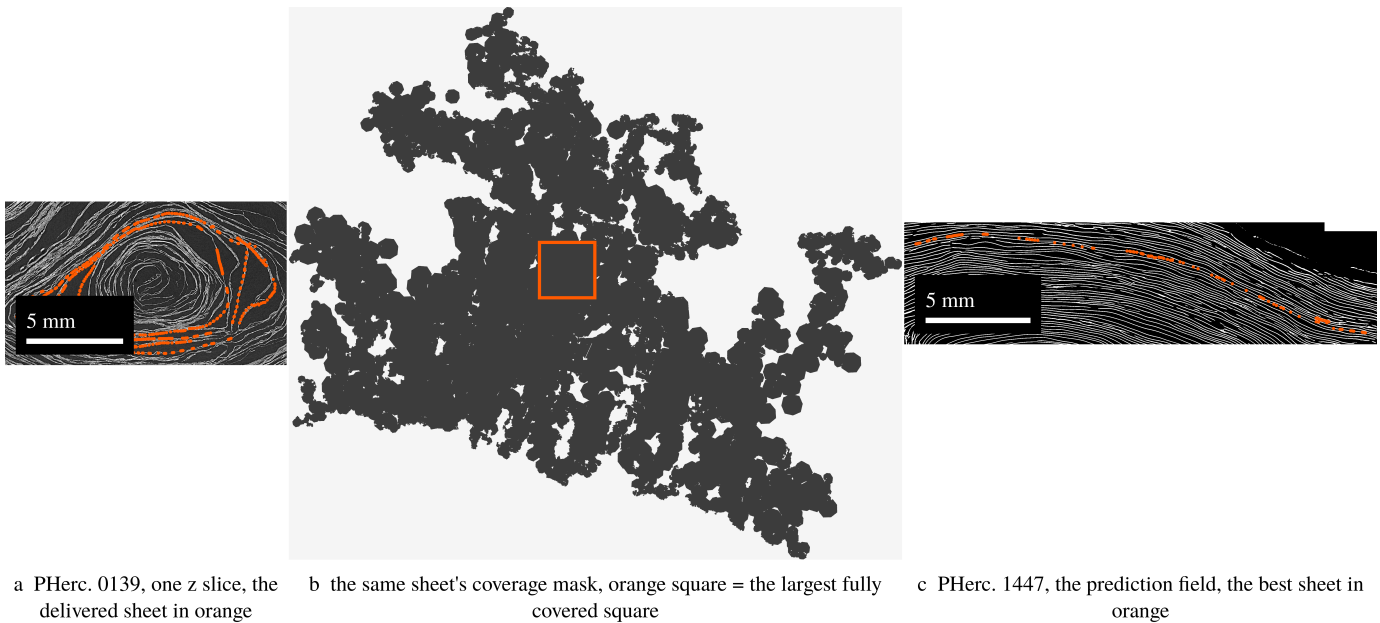


Fig. 5. **Papyrus, with the delivered sheet on it.** The material, with the square the chain gets out of it. Panel a is one z slice of the reference scroll's own grayscale at full resolution, read from the raw scan, with 606 cells of a delivered sheet lying on it. Panel b is that sheet's coverage mask and the largest fully covered square in it, 324 cells on a lattice of 3,054 by 3,389. That square is 12.123 mm at 9.362 μm a voxel, the largest over 610 measured sheets of twelve arms. Panel c is PHerC. 1447, whose grayscale is not on this disk, so the sheet is drawn on the published prediction field instead. The caption inside the figure says which of the two renderings each panel is. Its square is 12.7244 mm at 8.640 μm a voxel. The two panels of papyrus are different renderings of different scrolls and are not to be compared pixel for pixel. What they share is the square. The white bars in panels a and c are 5 mm.

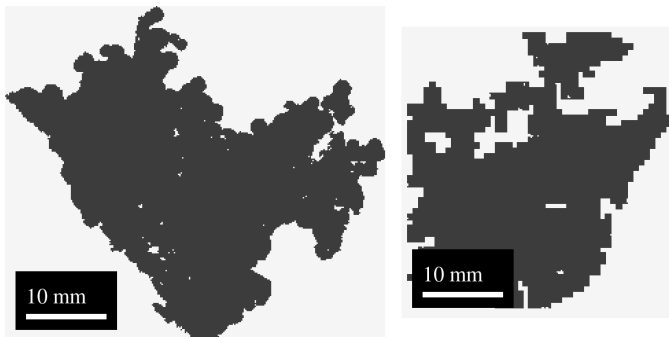


Fig. 6. **Our coverage mask beside a published segment's, at the same scale.** Our best sheet of PHerC. 1447 reaches 12.7244 mm, recomputed over the 100 sheets that the ten seed files of the search measured. The best published segment of the same scroll reaches 13.5262: the gap is 0.8018 mm, and this work claims nothing from it. The square is not drawn on the right panel, and the figure's own table says why in a row of its own. The segment's CSV measured a grid of 682 by 411 cells and the segment's tifxyz is 212 by 200, because the CSV read a different rendering of the same segment. A square placed on a grid it was not measured on would look like a measurement. Both panels are resampled to one millimetre per pixel, and the black bars are 10 mm.

on another. The factors of the quiet bench do not mix the two, and the ratios of the shared machine are named as such where they appear. Where a figure from the earlier shared machine is still given, the table it sits in says so on its own line.

Four further limits have nothing to do with time. The first is the number of ribbons. The kinds of section 2 are nine growth trees of one scroll from one binary route, and only three of them are ribbons. Every threshold, band and ordering that rests on the ribbon group therefore rests on three points.

The second is that nine seeds are not the distribution. The maximum of nine is not the maximum of the draw.

The third is that the guard repairs the consumer, not the producer. Section 5 says the fault is in the growth's bookkeeping. Whether the producer should be repaired instead is a decision with its own declaration, and this work does not take it.

The fourth is that identity is not correctness. Every output of the changed arm is byte for byte the output of the unchanged one, and that is the whole of the claim. Whether the stage's answer is right is a different question and is not touched here.

VIII. Open questions

This section names the questions the evidence above raises and does not answer, each with what would settle it.

Does the annealing make the sheets better or worse? Put back with the bridges, it changes the delivered sheets, but each run is a new draw (section 6). With a seed read from the environment the draws become repeatable, and the chain with and without the annealing becomes a comparison instead of a draw.

Does the split between ribbons and tangles hold beyond three ribbons? The kinds of section 2 rest on nine growth trees of one scroll, of which only three are ribbons, so every threshold, band and ordering drawn from the ribbon group rests on three points (section 7). More ribbon trees grown and measured in the same way would say whether the split and the cost of the $\subset$ stage on ribbons stay where they are.

IX. Conclusion

Two changes inside the bad patch finder, S1 and S2, make its $\subset$ stage finish on the tangles of PHerC. 1447, where it used to be abandoned. On the quiet bench the stage on the tangle seed40 is about two hundred times faster. Every file the five

delivery stages write is byte identical with and without the changes on fifteen named trees of two scrolls, and so is the stage's whole standard output on fourteen of them (section 4). The guard, S3, turns an abort into a result on the one tree whose alignment file names patches with no file. Its cost in time on ribbons is more than covered by the two changes (section 5).

For someone running the chain, this changes how long a seed takes and not what it delivers. A tangle now costs its growth and about a minute of $\circ$ stage rather than hours, so a search over many seeds becomes affordable. The squares, the defects and the distance to a 20 mm square stay exactly as they were (section 6). The three changes are proposed upstream as one pull request, stacked on the earlier performance patch they need (section 1).

About the author

Giovanni Pellerano is a computer engineer and whistle-blowing hacktivist. Co-author and Project Lead of the free and open-source software, GlobaLeaks, he is responsible for software development and for the support of the community that has grown around it worldwide. He is a digital rights advocate currently learning more about mainstream generative AI development and how to resist against their various and profound harms.

He had not heard of the Vesuvius Challenge before Saturday 5 September 2026. Everything reported in this series was done after that date, which is stated because it bounds what the work can be assumed to rest on: no prior familiarity with the data, the tools, the community or the literature, just a curious and hyperfocused mind joining this new journey.

Declaration of generative AI in the research and writing process

During the preparation of this work the author used Claude Fable 5.1 and Claude Opus 5.5 (Anthropic) in order to carry out the full research and to write the article: the models read the data, wrote and ran the tools, performed the measurements, and drafted the text. The author set the direction, supplied the domain judgement, challenged the conclusions, reviewed and edited the content, decided what was published, and takes full responsibility for the content of the publication.

The failure this arrangement is prone to is asserting a conclusion that no measurement supports. It happened repeatedly here and the corrections are in the text rather than removed from it: for example, a factor of seventeen claimed by an earlier prepared text is withdrawn in this series because it cannot be read back from anything on this disk.

What keeps the arrangement checkable is in the folder and not in this paragraph. The criterion of a study is registered before the run, with the day and the hour it was written, and is not revised once the results are in: the pre-registrations this article rests on are in `src/prereg/`, with their digests. Most numbers printed here expand a macro that `src/tools/paper_numbers.py` computes from a file in `src/evidence/`, and each of those files is written by a tool in `src/tools/`. The rest are copied by hand from a cell of such a file. A reader who doubts a number is not asked to trust the arrangement: the file, the tool and the command that produced it are in the same folder.

References

- [1] Vesuvius Challenge, *Open Problems: Why Reading Every Herculeum Scroll Is Still a Challenge*, 2026, read 30 September 2026. https://scrollprize.org/2026_open_problems
- [2] W. Stevens, *scrollreading*: flood-fill autosegmentation and surface growing. <https://github.com/WillStevens/scrollreading>
- [3] G. Pellerano, “An Automatic Patch Based Unwrapping Pipeline Applied to PHerc. 0826,” results/f49f6c7d-pherc0826-chain, *vesuvius-challenge-pipeline*, 2026.
- [4] W. Stevens, *report12.pdf*, a progress report on the scrollreading pipeline (bad patch detection, bridges and simulated annealing), 18 July 2026, in the repository at commit 62cbc21. <https://github.com/WillStevens/scrollreading>

Appendix: where each number comes from

Each [□] marker in the text stands for the file and column the number beside it was read back from.

marker	file and column
1	growth-bookkeeping/evidence/upstream-vs-ours.csv
2	a8-identity/evidence/identity-totals.csv, columns files_compared and files_differing, row both trees; the per file detail in identity.csv and the printed output in stdout-identity.csv of the same study
3	quiet-bench/evidence/c-stage-runs.csv, column seconds, tag c2
5	seed-search-1447/evidence/per-seed.csv, column growth_wall_clock_seconds
6	seed-search-1447/evidence/per-seed.csv, one row per seed; seed11 apart by columns growth_return_code and in_the_distribution
7	seed-search-1447/evidence/alignment-fanout.csv, column fan_out, computed from each growth's own rel.csv
8	early-tangle/evidence/prediction-seed44.csv, columns predicted, elapsed_seconds_when_read and share_of_first_100000
9	early-tangle/evidence/repeat-control-seed44.csv
10	seed-search-1447/evidence/per-seed.csv, column largest_square_mm_min_step, with the kind from alignment-fanout.csv, column fan_out
11	seed-search-1447/evidence/prediction-seed44-square.csv, columns claim and confidence
12	c-stage-cost/evidence/runs.csv, tag baseline-profile, columns seconds and return_code; the profile itself, with the shares and not the duration, is c-stage-cost/evidence/profile-baseline.csv
13	c-stage-cost/evidence/text-section-identity.csv
14	c-stage-cost/evidence/step-counts-model.csv
15	c-stage-cost/evidence/totals.csv, the two rows odometer steps, all four rounds, model and ... measured, unpruned
16	c-stage-cost/evidence/runs.csv, tag plain-rounds123, return_code 124
17	totals.csv
18	c-stage-cost/evidence/patch-verification.csv, column B_equals_C, yes, both sides db825fd1...
19	c-stage-cost/evidence/step-counts-measured.csv, variants pruned and unpruned, totals.csv; the kept chains per length on seed26 from column sequences
20	c-stage-cost/evidence/phases.csv, summed by tool into totals.csv
21	quiet-bench/evidence/c-stage-summary.csv, columns every_second_that_finished, spread_seconds, spread_pct and cores_busy_min
22	quiet-bench/evidence/c2-factor.csv, column factor
23	seed-search-1447/evidence/alignment-fanout.csv, columns patch_files and edges
24	seed-search-1447/evidence/untouched-seed11-downstream.csv, columns seconds and return_code, row c
25	c-stage-cost/evidence/runs.csv, tag c2-11-35, stage c
26	c-stage-on-0139/evidence/trees.csv, column fan_out
27	c-stage-on-0139/evidence/c-stage-times.csv, columns median_seconds, min_seconds, max_seconds and cores_busy_min/cores_busy_max, 0.0 to 12.1 of 24
28	c-stage-on-0139/evidence/binaries.csv, column corrections_present
29	c-stage-on-0139/evidence/c-stage-ratios.csv, columns ratio_median_over_median and ranges_overlap
30	c-stage-on-0139/evidence/pristine-check.csv, column unchanged, eight rows, all yes
31	c-stage-cost/evidence/stdout-identity.csv, column identical, twelve rows, all yes; the longest output is the maximum of column reference_lines over those twelve rows
32	c-stage-on-0139/evidence/stdout-identity.csv, sixteen rows, all yes
33	quiet-bench/evidence/a8-factor-medians.csv, column factor
35	orphan-guard/evidence/runs.csv, log/chain-plain-PHercl447-seed34-c.txt
36	badpatch-crash/evidence/binary-gate.csv

continued on the next page

marker	file and column
37	badpatch-crash/evidence/crash-diagnosis.csv, columns value, out_of and the_check, written by that study's tools/crash_diagnosis.py from the instrumented run's log; the distinct patches against key-containers.csv, whose column times_reported_missing sums to the failing chains
38	badpatch-crash/evidence/orphans-per-tree.csv, columns ids_in_rel_with_no_patch_file and of_those_reachable_as_second_of_a_kept_length_2_chain
39	orphan-guard/evidence/refused-chains.csv, columns chains_refused, patches_involved, predicted_patches, matches_prediction
40	orphan-guard/evidence/cross-check.csv
41	orphan-guard/evidence/runs.csv, log/memory-watchdog.txt
42	orphan-guard/evidence/skipped-chains.csv
43	orphan-guard/evidence/sheets-identity.csv, orphan-guard/evidence/all-outputs-identity.csv
49	growth-bookkeeping/evidence/id-inventory-seed34.csv
54	seed-search-1447/evidence/per-seed.csv, column largest_square_mm_min_step, maximum over the nine rows with in_the_distribution yes
55	published-segments-eligible/evidence/segments-eligible.csv, column square_mm_min_step, maximum over the fifteen PHerc. 1447 rows, segment 20251105093211-z_dbg_gen_00320, at that scroll's own 8.640 um voxel
56	per-seed.csv, column sheets_reaching_20mm
57	segments-eligible.csv, column reaches_20mm, no on every row
63	stevens-remedy-half-on/evidence/remedy-parts.csv, columns file, line_62cbc21, quote, default_in_chain and house_delivered_source, rows M1a to M3c; every quotation checked verbatim at its line of git show 62cbc21 by tools/studies/stevens-remedy-half-on/remedy_parts.py
64	stevens-remedy-half-on/evidence/per-seed.csv, columns status and delivered_sheets_changed, ten rows, written by summarise.py; the seeds from ten-seeds.csv (draw_seeds.py); the identity with the annealing left out from identity-off-PHerc1447-seed262.csv, row verdict (tools/studies/patch-filter-harness-1447/identity_compare.py); the runner tools/studies/stevens-remedy-half-on/rerun_remedy.sh
65	stevens-remedy-half-on/evidence/draws-per-seed.csv, columns best_square_mm_before, best_square_mm_d0 to d3, best_square_mm_spread_d0_d3 and excluded_jaccard_mean_d1_d3; overlap-per-seed.csv, columns klass and E per seed and draw, the rows SEED and the rule in its header; both written by tools/studies/stevens-remedy-half-on/summarise_draws.py
68	src/evidence/manifests/PHerc1447.json, column voxel_um, a copy of pipeline/dataset_s/manifests/PHerc1447.json of the laboratory
69	c-stage-cost/evidence/runs.csv, column seconds; the load of the shared machine from column cores_busy_at_start
71	seed-search-1447/evidence/per-seed.csv, column largest_square_mm_min_step; the maximum, median and minimum are the rows of seed-search-1447/evidence/summary.csv, column value
72	seed-search-1447/evidence/per-seed.csv, column sheets_reaching_20mm, 0 on all three rows
73	seeds-at-scale-1447/evidence/seed-rule.csv, column passes_rule
74	seed-search-1447/evidence/alignment-fanout.csv, model column chain_cost_at_length_5
75	c-stage-cost/evidence/c-stage-times.csv, columns reference_seconds, arm_seconds and speedup; the ratios of the shared machine are its column speedup
76	quiet-bench/evidence/throttle-overlap.csv
77	c-stage-on-0139/evidence/c-stage-diff.csv, column verdict
78	c-stage-on-0139/evidence/identity-by-tree.csv, columns carried_input and written_by_downstream
79	quiet-bench/evidence/a8-factor-medians.csv, a median against a median, and a8-factor.csv beside it, written by tools/summarise_c.py, the smallest run of each arm
80	the study growth-bookkeeping/, read from the source and the filesystem with no run

continued on the next page

marker	file and column
83	seed-search-1447/evidence/summary.csv, the gap and the repeat spread; neither is in the files of markers 54 and 55, which carry the squares they are computed from

Appendix: what each figure is drawn from

Each figure is drawn by the script named in it, from the CSV of plotted numbers beside it. No number in a figure is typed by hand.

figure	what it shows	drawn from
Fig. 1	A sheet the old stage abandoned, on the scroll it was grown in.	Two slices of the raw masked scan at its level 3, one pixel eight voxels, with the sheets' cells on them (evidence/figures/s-f7-a-sheet-the-old-stage-abandoned.csv), whose rows name the trees, the sheets, their sha256 and the display window.
Fig. 2	The odometer's steps, modelled against counted.	Bars on a logarithmic axis, one seed each (evidence/figures/s-f2-odometer-steps.csv, columns modelled_steps and counted_pruned_steps).
Fig. 3	The c stage, untouched against changed, on the quiet bench.	One pair of bars per seed on a logarithmic axis, the factor written over each pair (evidence/figures/s-f1-c-stage-before-after.csv, columns quiet_plain_seconds, quiet_c2_seconds and quiet_factor_label); the identity marks from its identity_ columns, read from c-stage-cost/evidence/identity.csv, stdout-identity.csv and c-stage-on-0139/evidence/identity-by-tree.csv. The right panel is the reference scroll, drawn as medians with the observed range from c-stage-on-0139/evidence/c-stage-times.csv.
Fig. 4	Two runs mixed in one folder.	Patch ids on the x axis, the stale ones marked (evidence/figures/s-f4-seed34-two-runs-mixed.csv, columns stale_patch_id, comp_label and comp_count).
Fig. 5	Papyrus, with the delivered sheet on it.	Three panels of equal height, each captioned inside the figure (evidence/figures/s-f5-papyrus-with-the-sheet.csv), whose rows also carry the display window of each grayscale panel.
Fig. 6	Our coverage mask beside a published segment's, at the same scale.	Two panels (evidence/figures/s-f6-coverage-beside-a-published-segment.csv).